Blockchain Lab3 report

Setup:

Part - A Proof of Work

The statistics are the lesson:

```
================================================================================
PART A -- Proof of Work: parallel mining and difficulty retargeting
================================================================================
Retargeting arithmetic (no mining involved):
  window took half the target time -> 2000  (difficulty up)
  window took twice the target time -> 500  (difficulty down)
  window took 1/100th of the target -> 4000  (clamped at 4x)
  window took 100x the target       -> 250  (clamped at 1/4)

  The clamp bounds one adjustment to a factor of four either way. Without
  it, a single manipulated or anomalous window could move difficulty to a
  value the chain cannot mine its way out of.

Mining 60 blocks (retarget every 10, target 0.30 s/block)...

blocks mined        : 60
target interval     : 0.300 s
mean interval       : 0.184 s
median interval     : 0.136 s
stdev of intervals  : 0.106 s
min / max interval  : 0.087 / 0.530 s
difficulty start/end: 1 -> 11
median / mean       : 0.736   (exponential predicts ~0.693)
stdev / mean        : 0.573   (exponential predicts ~1.0)

chain height  : 60
cumulative work: 17104896

  Median well below the mean, and a standard deviation close to the mean,
  are the fingerprints of an exponential distribution. They follow from the
  search being memoryless: no attempt brings you closer to a solution, so
  a small miner's expected share equals its hash-rate share exactly.

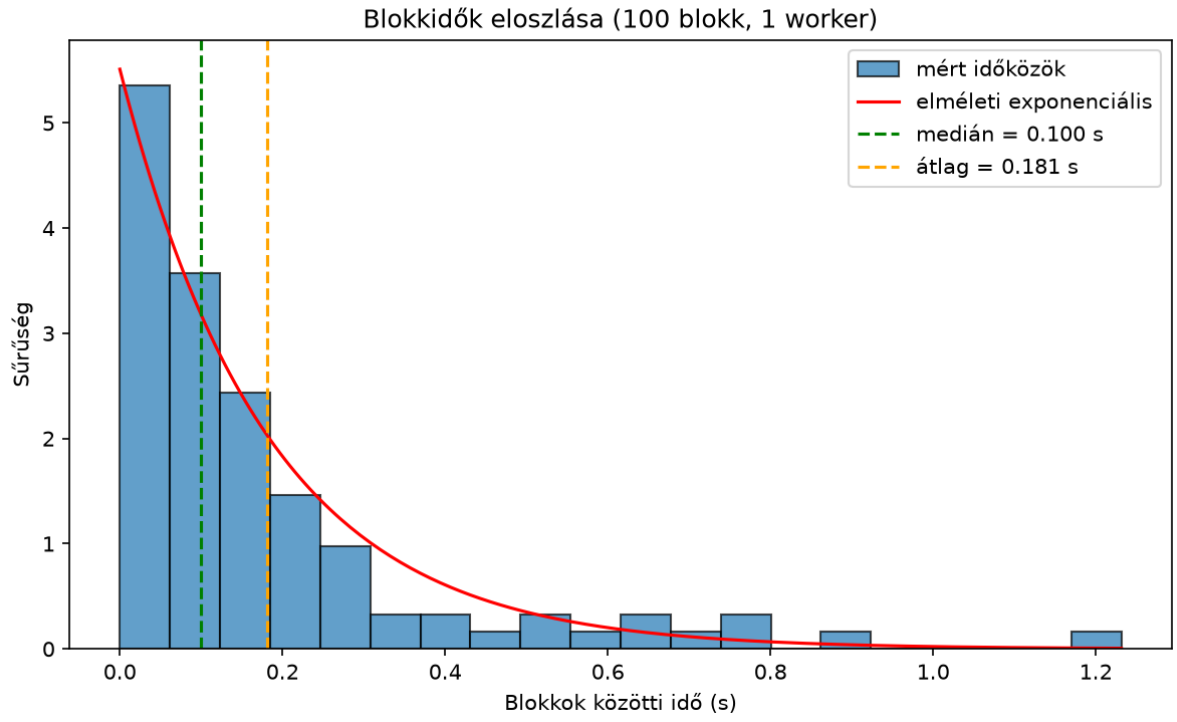
  This is also why a two-hour gap between Bitcoin blocks is not a fault.
```

blocks	median/mean	stdev/mean
60	0.736	0.573
70	0.701	0.765
80	0.747	0.899
100	0.677	1.096

Parallel mining:
median/mean: 0.587
stdev/mean: 1.055
min/max interval: 0.000 / 1.318 s

I mined 100 blocks with a single worker. The median/mean ratio was 0.587 (theory: 0.693) and the stdev/mean ratio was 1.055 (theory: 1.0). The shortest interval was 0.000 s and the longest 1.318 s, a difference of more than an order of magnitude. Both ratios fall within the expected ranges (0.60–0.80 and 0.80–1.20). The small deviation from theory comes from the sample size: with 100 blocks, one or two unusually long intervals noticeably shift the mean.

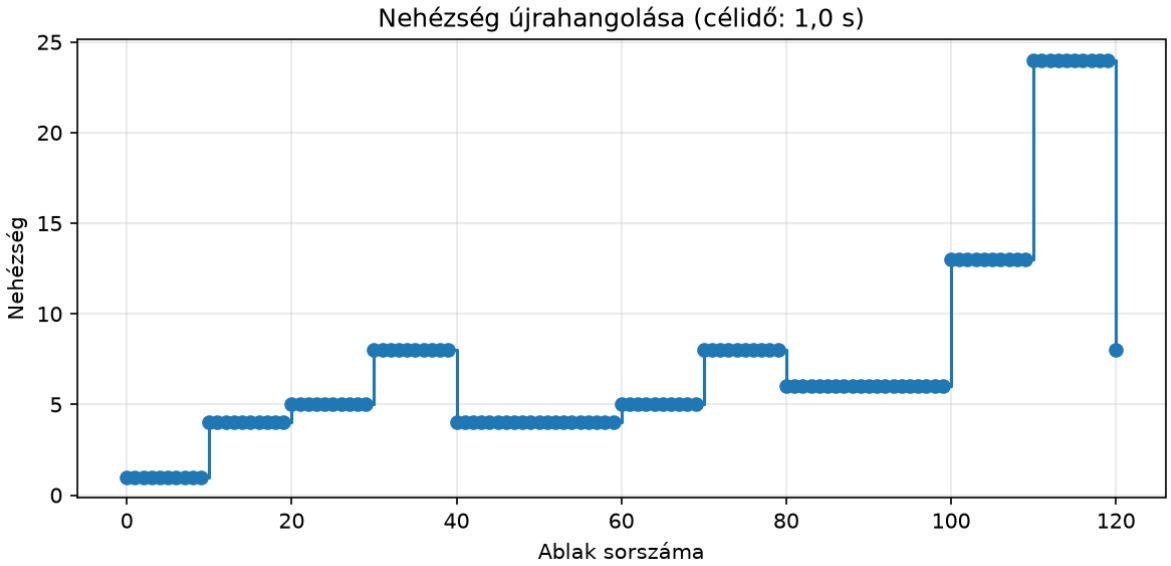
The histogram is strongly right-skewed. The tallest bar is the leftmost bin, containing the shortest intervals, and the frequency falls off roughly geometrically to the right. The distribution is not a bell curve centred on the mean but exponential: short intervals are the most common, and long ones are rare but do occur. The measured histogram fits the theoretical exponential density well. Memorylessness means that earlier failed attempts bring a miner no closer to a solution: if a miner has been searching unsuccessfully for a while, its expected remaining time is the same as if it had just started. This makes mining a fair lottery: every miner wins in proportion to its hash rate. A miner with 10% of the network's hash power finds about 10% of the blocks.



If the puzzle were not memoryless, meaning accumulated work counted (like a race where a fixed amount of work must be completed), the largest miner, which accumulates work fastest, would almost always finish first. It would win disproportionately more blocks than its hash-rate share, and smaller miners would almost never win. Small miners would drop out, mining would concentrate in the hands of one or two participants, and this would undermine the system's decentralisation and security.

Memorylessness means that earlier failed attempts bring a miner no closer to a solution: if a miner has been searching unsuccessfully for a while, its expected remaining time is the same as if it had just started. This makes mining a fair lottery: every miner wins in proportion to its hash rate. A miner with 10% of the network's hash power finds about 10% of the blocks.

If the puzzle were not memoryless, meaning accumulated work counted (like a race where a fixed amount of work must be completed), the largest miner, which accumulates work fastest, would almost always finish first. It would win disproportionately more blocks than its hash-rate share, and smaller miners would almost never win. Small miners would drop out, mining would concentrate in the hands of one or two participants, and this would undermine the system's decentralisation and security.



With a 1.0 s target, difficulty moved between about 1 and 3 across the twelve windows. It rose after fast windows and fell after slow ones. This is negative feedback: the system continuously corrects toward the target time, so difficulty oscillates around an equilibrium instead of climbing steadily. The oscillation happens because block times are random: any given window can be faster or slower by chance.

At the default 0.30 s target, difficulty stays at 1 for the whole run, because even difficulty 1 takes the Python miner longer than 0.30 s. Every window is "too slow", so difficulty would go down, but it cannot go below 1.

Call	Result	Explanation
`retarget(1000, 300, 600)`	2000	the window was twice as fast as it should have been, so difficulty doubles
`retarget(1000, 1200, 600)`	500	the window was twice as slow, so difficulty halves
`retarget(1000, 6, 600)`	4000	would be ×100, but the clamp limits it to ×4
`retarget(1000, 60000, 600)`	250	would be ÷100, but the clamp limits it to ÷4

Part B – Proof of Stake

```
================================================================================
PART B -- Proof of Stake: Casper FFG finality and slashing
================================================================================
>>> Scenario 1: honest majority reaches finality
  epoch 1: justified=(1, root-1)  finalized=None
  epoch 2: justified=(2, root-2)  finalized=(1, root-1)
  epoch 3: justified=(3, root-3)  finalized=(2, root-2)

  (2, root-2) is final. Two phases were needed: a checkpoint is
  justified by a 2/3 link, and finalised only once its successor is also
  justified. That second phase is why Ethereum finality takes ~12.8 minutes.

>>> Scenario 2: a single validator equivocates (double vote)
  validator 63 balance : 32.0000 -> 29.5916 ETH
  slashed              : True
  loss                 : 2.4084 ETH

  A lone equivocator -- typically a misconfigured setup running one key on
  two machines -- loses a small fraction. The network was never at risk.

>>> Scenario 3: a coordinated attack (many validators equivocate together)
  20 validators slashed simultaneously
  each balance         : 32.0000 -> 2.5680 ETH
  loss per validator   : 29.4320 ETH

  Same offence, vastly larger penalty. The correlated term scales with the
  fraction of stake that misbehaved at once, so punishment tracks SYSTEMIC
  RISK rather than individual error. Accidents stay cheap; attacks do not.

>>> Scenario 4: a surround vote
  validator 5 slashed  : True
  rule                 : surround_vote

  Harder to spot than a double vote: one vote's source-target span strictly
  encloses the other's, asserting two incompatible histories over the same
  range. Both are cryptographically provable from the signatures alone.

>>> Scenario 5: the inactivity leak
  epoch  1: finalized=None            leaked=  0.000 ETH  voting share=37.5%
  epoch  5: finalized=None            leaked= 20.000 ETH  voting share=37.9%
  epoch  8: finalized=None            leaked= 19.077 ETH  voting share=39.0%
  epoch 11: finalized=None            leaked= 18.197 ETH  voting share=40.1%

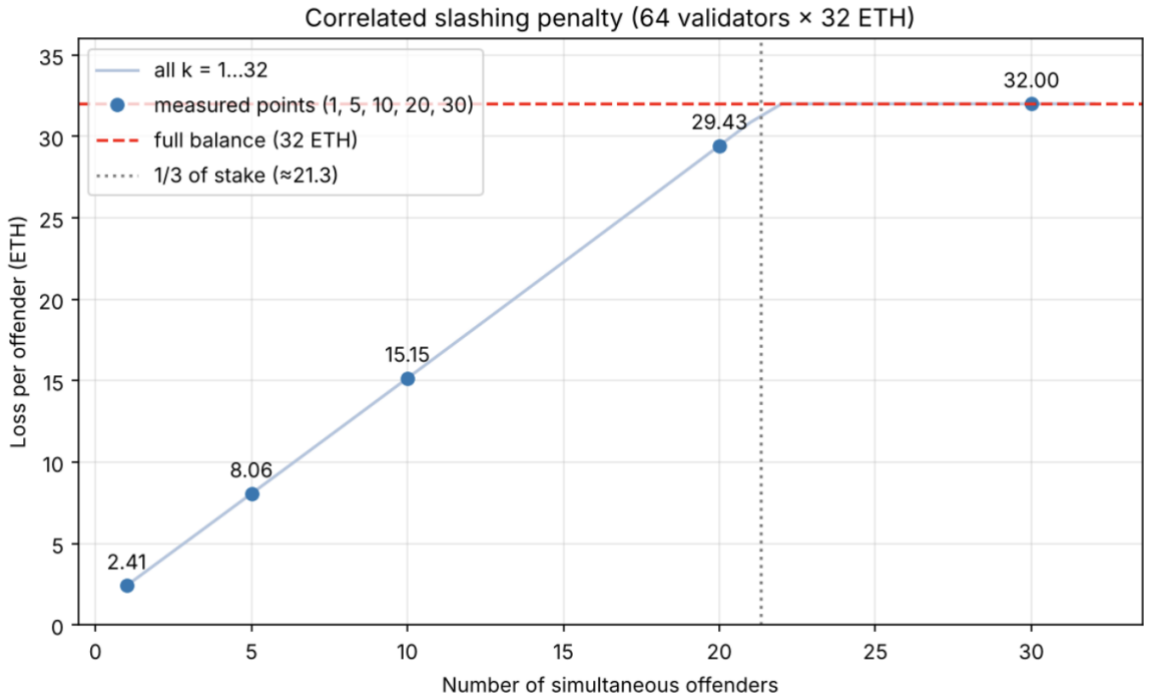
  Casper chooses safety over liveness: without a 2/3 quorum it simply stops
  finalising. The leak restores liveness by shrinking absent validators'
  stake until the participating remainder passes 2/3 again. Honest nodes
  stranded by a partition also lose -- that is the price of recovering.

epoch              : 11
validators         : 64 (64 active, 0 slashed)
total active stake : 1914.39 ETH
2/3 threshold      : 1276.26 ETH
current justified  : (0, genesis)
finalized          : (0, genesis)
```

Scenario	Your result
1 — finality	epoch 2 finalises (1, root-1), epoch 3 finalises (2, root-2)
2 — lone equivocator	32.0000 → 29.5916, loss 2.4084 ETH
3 — 20 offenders	32.0000 → 2.5680, loss 29.4320 ETH
4 — surround vote	slashed: True, rule: surround_vote
5 — inactivity leak	no finalisation, 37.5 % → 40.1 %

loss per validator table and plot:

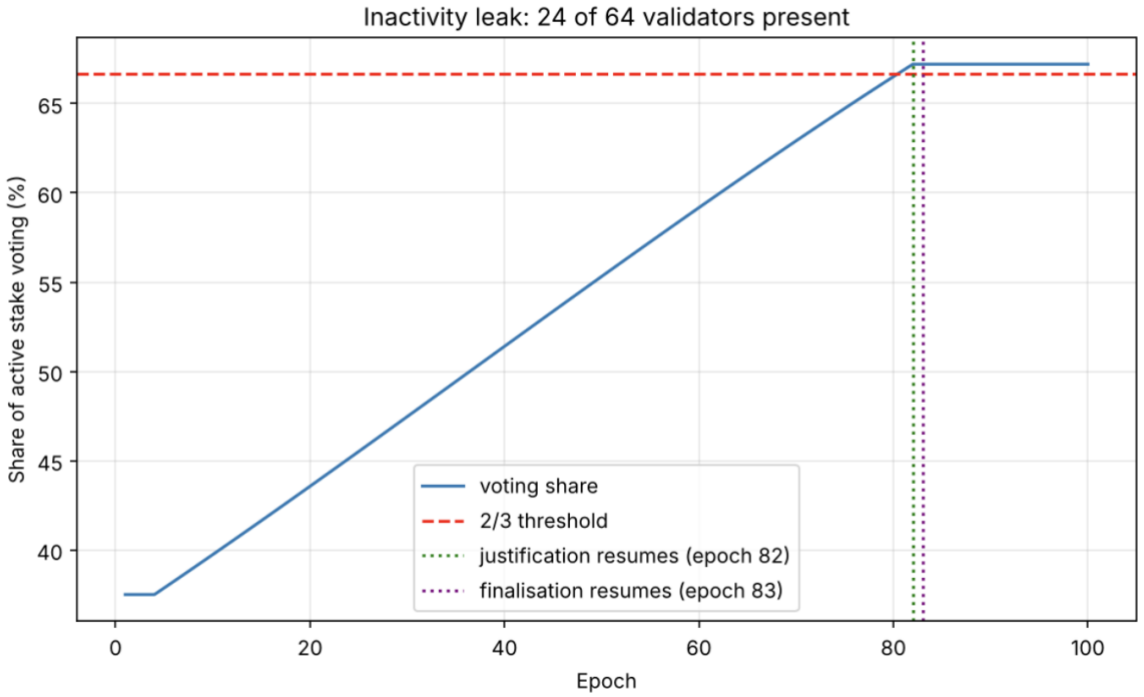
Offenders	1	5	10	20	30
Loss (ETH)	2,41	8,06	15,15	29,43	32,00



The loss rises in a straight line, about 1.4 ETH per extra offender, then flattens at 32 ETH. Every offender first loses a fixed 1 ETH (1/32 of the stake), plus a correlated penalty of the remaining balance × 3 × slashed/total. Once about a third of the stake is slashed together (22 of 64 validators), that factor hits 1 and everyone loses their whole balance.

I think it's fair. The knee is exactly at the 1/3 Byzantine threshold. Below it, the offenders couldn't have finalised conflicting checkpoints, so it was most likely an accident and the penalty is small. Above it, the same offence could break safety, so the penalty is total. The downside is that an honest validator who makes a mistake during an attack gets punished like an attacker. But that usually means it shared a client or host with others, which is a risk, so the penalty pushes validators to diversify.

Epoch	**1**	**11**	**20**	**40**	**60**	**80**	**82**
Vot	37,5 %	40,1 %	43,6 %	51,4 %	59,2 %	66,5 %	67,2 %



With only 24 of 64 validators voting (37.5 %), there's no 2/3 quorum, so finality stops. After 4 epochs the leak starts draining the absent validators, and the voting share climbs: 43.6 % at epoch 20, 51.4 % at 40, 59.2 % at 60. It passes 2/3 at epoch 82, justification resumes there, and finalisation follows in epoch 83. The original demo never justified anything because its votes came from unjustified checkpoints. The cost is that honest validators cut off by a partition lose stake too, but without the leak the chain could never recover.

Part C – Gossip netwroking and partition

```
============================================================================
PART C -- asynchronous gossip, network partition and reorganisation
============================================================================
Building 8 nodes, ring topology plus random edges.
Mining is modelled as an exponential delay; hashing itself is Part A.

  phase 1 (connected)  heights=[18] converged=True
  PARTITION  A=['node0', 'node1', 'node2', 'node3']  B=['node4', 'node5', 'node6', 'node7']
  phase 2 (split)      distinct tips=2 heights=[28, 30]
  HEAL
    just after heal    distinct tips=1 (a tie at equal work is resolved by the next block, not by th
e protocol)
  phase 3 (healed)     distinct tips=1 converged=True

node    height  tip           mined  recv  txs  reorgs
------------------------------------------------------------
node0       74  60d9f7ed55c1     20    65    1       0
node1       74  60d9f7ed55c1      0    85    2       0
node2       74  60d9f7ed55c1     24    61    2       0
node3       74  60d9f7ed55c1      0    85    2       0
node4       74  60d9f7ed55c1     21    65    1       1
node5       74  60d9f7ed55c1      0    86    1       1
node6       74  60d9f7ed55c1     34    52    1       1
node7       74  60d9f7ed55c1      0    86    1       1

messages delivered : 2796
messages dropped   : 203  (partitioned away)
consensus reached  : True

  reorganisations observed across all nodes: 4

  During the split each side built on its own tip, so both sides had a
  perfectly valid chain and neither was wrong. On healing, the heavier
  branch won and the lighter one's blocks were discarded -- taking their
  transactions back to the mempool.

  This is the CAP trade-off made visible. Proof of work chooses AVAILABILITY:
  both partitions kept accepting transactions, and consistency was restored
  only afterwards, by reverting one side. Casper FFG in Part B chooses the
  opposite: it refuses to finalise rather than risk contradicting itself.
```

Reorganization depths:

seed 1: reorganisations observed across all nodes: 6

seed 2: reorganisations observed across all nodes: 6

seed 3: reorganisations observed across all nodes: 6

1 {'node0': [1, 1], 'node1': [1, 1, 1], 'node2': [1, 1], 'node3': [1, 1, 1], 'node4': [1, 1, 1], 'node5': [1, 1], 'node6': [1, 1, 6], 'node7': [1, 1, 6], 'node8': [1, 1, 6], 'node9': [1, 1, 1, 6], 'node10': [1, 1, 1, 6], 'node11': [1, 1, 1, 6]}

2 {'node0': [1, 1], 'node1': [1], 'node2': [1, 1], 'node3': [1], 'node4': [1], 'node5': [1, 1], 'node6': [1, 1, 6], 'node7': [1, 1, 6], 'node8': [1, 6], 'node9': [1, 6], 'node10': [1, 6], 'node11': [1, 1, 6]}

3 {'node0': [1], 'node1': [1, 1, 1], 'node2': [1, 1], 'node3': [1, 1], 'node4': [1, 1, 1], 'node5': [1, 1, 1], 'node6': [1, 1, 6, 1], 'node7': [1, 1, 6], 'node8': [1, 1, 6], 'node9': [1, 1, 6], 'node10': [1, 1, 1, 6], 'node11': [1, 1, 6]}

I ran the gossip simulation with 12 nodes and seeds 1, 2 and 3. In all three runs, nodes 6–11 each had one reorganisation of depth 6, and every other reorganisation was depth 1. Seed 1 had a few more depth-1 reorgs than seed 2, but the pattern was the same each time.

The depth-6 reorgs happened at the heal. Side A had two miners (node0 and node4) and side B had only one (node8), so A's chain was heavier and B's blocks got thrown away. The depth-1 reorgs are just normal races where two blocks were found at almost the same time, and they happened on both sides.

Deeper reorganisations get more likely the longer the partition lasts, because both sides keep mining their own chain during the split. When the network heals, the losing side has to discard every block it mined since the split, so the reorg is as deep as the losing branch is long. That branch grows with time. In my runs the split lasted 1.5 s and B mined a block about every 0.25 s, which gives about 6 blocks, and that's exactly the depth I saw. If the split lasted twice as long, I'd expect a reorg of about 12.

My deep reorgs were 6 instead of the typical 2–5 mentioned for the default run, because with 12 nodes the losing side had one miner mining about 6 blocks during the 1.5 s split, and phase 2 sometimes showed 4 tips because blocks take longer to spread across 12 nodes.